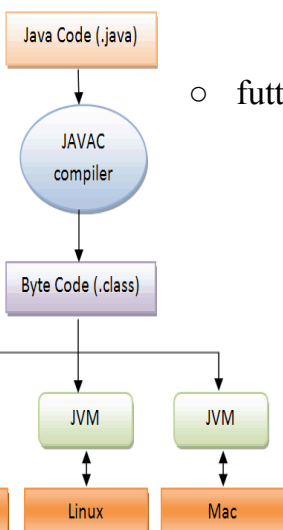


Java és C++ programok fordítása és futtatása. Parancssori paraméterek, fordítási opciók, nagyobb projektek fordítása. Absztrakt-, interfész- és generikus osztályok, virtuális eljárások. A virtuális eljárások megvalósítása, szerepe, használata

PROGRAMOK FORDÍTÁSA ÉS FUTTATÁSA

JAVA modell

- JAVA egy **menedzsel, platformfüggetlen nyelv**, amely az “egyszer megírni, bárhol futtatni” (Write Once, Run Anywhere) elvét követi
 - menedzsel, azaz nem közvetlen a hardveren fut, hanem a JVM-en, emiatt lehetséges a WORA is
 - a JAVA bájtkódja bitre pontosan ugyanaz Windowson és Linuxon is, míg a C++-t újra kell fordítani
- menete:
 - fordítási folyamat



- a forráskód (.java) a **JAVAC** fordító bájtkódra (.class) fordítja le
 - a bájtkód nem gépkód, hanem egy köztes nyelv
 - futtatás
 - bájtkód egy gépfüggetlen kód, amely nem a fizikai processzoron, hanem egy virtuális gépen fut (**JVM**)
 - **JVM (Java Virtual Machine)**
 - értelmezi a bájtkódot és végre is hajtja
 - minden operációs rendszerre van saját JVM, ami ugyanazt a bájtkódot tudja futtatni csak más OS környezetbe
 - **JIT (Just-In-Time) fordító**
 - a futtató környezet része
 - a futás során a bájtkód gyakran használt részeit natív (gépi) kóddá alakítja a gyorsabb végrehajtás érdekében
- ökoszisztéma
 - fejlesztéshez a JDK (Java Development Kit)
 - futtatáshoz a JRE (Java Runtime Environment)
 - JVM-et és az alapvető osztálykönyvtárakat tartalmazza (API)

C++ modell

- a C++ egy **natív, platformfüggő nyelv**, amely közvetlenül a célhardver gépi kódjára fordítódik (**hardverközeli működés**) a **maximális teljesítmény** érdekében
 - **natív kód**, azaz az eredményül kapott futtatható fájl közvetlenül a processzor utasításkészletét használja
 - ezért gyorsabb mint a JAVA, de csak azon a platformon (OS és hardver) fut amelyre fordították
 - **platformfüggő** mivel, nincs virtuális gép, a program közvetlenül éri el a hardvert és a memóriát, de minden célplatformra külön kell lefordítani a forráskódot

- **fordítási szakaszok**
 - **előfeldolgozás (preprocesszor)**
 - feldolgozza a header fájlokat (#include direktívákat) és a makrókat (#define)
 - rendezzi a program sorait és a kommentek helyettesíti whitespace-ekkel
 - **fordítás (compiler)**
 - forráskódból (.cpp) először assembly kódot gyárt, majd ebből tárgykódot (object code, .o vagy .obj) készít
 - ez már gépi kód, de még hiányos (nincsenek feloldva a külső hivatkozások pl: function hívásnál, helyükre placeholder kerül)
 - szimbólum táblát tartalmazza
 - **exportok:** függvények, változók amik ebben az osztályban definiáltak és mások is használhassák
 - **importok:** olyan nevek (függvények, változók), amiket ebben a fájlban használtál, de nincsenek itt definiálva
 - **szerkesztés (linker)**
 - összefűzi a különálló tárgykódokat és a könyvtárakat
 - feloldja a címhivatkozásokat
 - egyetlen végrehajtható bináris fájlt állít elő
- .cpp + .h -> preprocesszor -> fordító -> .obj fájlok -> linker -> .exe / bináris

PARANCSORI PARAMÉTEREK ÉS FORDÍTÁSI OPCIÓK

Parancssori paraméterek kezelése

- parancssori paraméterek lehetővé teszik, hogy indításkor (pl: terminálból) adatokat adjunk át a programnak a forráskód módosítása nélkül
- **JAVA modell**
 - belépési pont a main függvény: public static void main(String[] args)
 - static-nak kell lennie különben nem lehetne példányosítás nélkül elérni
 - **args** egy String tömb, mely a paramétereket tárolja
 - args[0] az első tényleges paraméter
 - paraméterek számát az args.length-el érjük el
- **C++ modell**
 - belépési pont a main függvény: int main(int argc, char* argv[])
 - argc: az átadott paraméterek száma (egész szám)
 - legalább 1, mert a program neve mindig benne van argv-ben
 - argv: karaktertömbökre (C-stílusú stringek) mutató mutatók tömbje
 - argv[0] mindig a program nevét, vagy az indító útvonalat tartalmazza
 - erre azért van szükség, mert C++-ban a program binárisa nem tudhatja pontosan milyen néven hívták meg, JAVA esetén meg az osztálynév indításkor (java MyClass) már ismert
 - felhasználó által ténylegesen megadott első paraméter az argv[1]

Fordítási opciók (JAVA)

- a **javac** parancs segítségével történik a fordítás
- fontosabb kapcsolók
 - **-sourcepath <útvonal>**
 - megadja, hol keresse a forrásfájlokat
 - **-d <könyvtár>**
 - megadja a célkönyvtárat, ahová a lefordított .class bináris fájlok kerülnek
 - amennyiben csomagokat, packages-t használunk a megfelelő mappaszerkezetet a fordító hozza létre
 - **-cp <könyvtár> vagy -classpath <könyvtár>**
 - megadja, hogy a fordító hol keresse a külső osztályokat vagy könyvtárakat (.jar fájlokat)
 - **-version**
 - kiírja a használt JDK verziót
 - **-Werror**
 - figyelmeztetés esetén megáll a fordítás

Fordítási opciók (C++)

- a **g++ (GCC)** fordító leggyakoribb kapcsolói
 - **-o <fájlnév>**
 - meghatározza a kimeneti (futtatható) fájlnévét
 - ha elhagyjuk, a kimenet a.out (Linux) vagy a.exe (Windows) lesz.
 - **-c**
 - csak fordítást végez, de nem futtatja a szerkesztőt (linkert)
 - az eredmény így egy tárgykód (.o vagy .obj fájl) lesz
 - NAGYOBB PROJEKTEKNÉL FONTOS
 - **-Wall**
 - bekapcsolja az összes fontos figyelmeztetést (warnings), ezzel segítve a hibák korai kiszűrését
 - **-g**
 - hibakeresési információkat (debug symbols) ad a fájlhoz, így a program nyomon követhető például a gdb segítségével
 - **-O2 (vagy -O3)**
 - optimalizálás bekapcsolása
 - fordító hatékonyabbá teszi a gépi kódot a futási sebesség érdekében
 - **-std=c++17**
 - megadja a használt nyelvi szabványt (pl. C++11, C++17, C++20)

Nagyobb projektek fordítása és kezelése

- nagyobb projektek esetén a cél a kód átláthatósága, az újrafelhasználhatóság és a névütközések elkerülése
- **Modularitás JAVA-ban**
 - JAVA a könyvtárstuktúrára épülő **csomagkezelést (packages)** használja a modularitás alapjaként
 - **csomagok (packages)**
 - forrásfájlok elején a package kulcsszóval jelezzük, hogy hova, melyik csomagba tartozik az osztály
 - pl: package hu.szte.prog;
 - fájlrendszert is meghatározza, azaz a mappaszerkezetet (így kell kinéznie)
 - pl: a fenti példát követve: hu/szte/prog
 - **láthatóság**
 - a csomagok segítik a hozzáférés szabályozását
 - a default, azaz a módosító nélküli láthatóság azt jelenti, hogy package-private, azaz csak azonos csomagban lévő osztályok érik el a tagokat
 - **importálás**
 - más packagek-et is tudunk importálni az import kulcsszóval, ezzel az adott csomagban lévő osztályokat tudjuk elérni
 - **JAR (Java Archive)**
 - nagyobb projektek esetén a lefordított .class fájlokat egyetlen tömörített .jar állománya csomagoljuk
 - ez megkönnyíti a terjesztést és a futtatást is
 - **Build eszközök**
 - komplex projekteknél már nem kézzel hívjuk a javac-ot, hanem olyan eszközöket használunk, mint a **Maven** vagy a **Gradle**
 - ezek kezelik a függőségeket (külső könyvtárakat) és a fordítási sorrendet
- **Modularitás C++-ban**
 - C++-ban a modularitás alapja a **deklaráció** és a **definíció szétválasztása**, valamint a névterek használata
 - vannak **header** (.h, .hpp) és **forrás** (.cpp) fájlok
 - a **header** fájlok tartalmazzák a deklarációkat (interfész)
 - mit tud az osztály, mik a metódusok szignatúrái
 - a **forrás** fájlok pedig az implementációt tartalmazzák
 - hogyan működnek a metódusok
 - **különálló fordítás**
 - minden .cpp fájlt külön-külön fordít le a fordító tárgykóddá (.o, .obj)
 - előnye, ha csak 1 fájl módosul elég azt újrafordítani

- **szerkesztő (linker) szerepe**
 - ő fűzi össze a különálló tárgykódokat, melyeket a fordító ad és feloldja a hivatkozásokat (pl: ha az A.cpp hív egy függvényt, ami a B.cpp-ben van) is
- **Header Guard (védőfeltétel)**
 - header fájlokba vannak azok a deklarációk és definíciók amiket használnánk
 - header fájlok egymást is használhatják ezért előállhat az, hogy egy header fájlt többször include-olunk
 - ez gond lehet ha a header tartalmaz makrót vagy typedefeket mivel ezek nem fordulhatnak elő többször
 - meg kell akadályozni a többszörös deklarációt
 - megoldás rá különböző makrók használata vagy a `#pragma once` (mindkettő ugyanazt csinálja és támogatott is)
 - **makró**
 - ha több ugyanolyan nevű header fájl van különböző könyvtárakba akkor az problémát okozhat, mivel a makró megegyezhet
 - `#ifndef KUTYA_H`
`#define KUTYA_H`
`class Kutya { /* ... */ };`
`#endif`
 - **pragma**
 - nem része a C++ szabványának, de a legtöbb fordító támogatja
 - header fájl első sorába kell írni
 - azt mondja meg, hogy "csak egyszer vegye figyelembe"
 - `#pragma once`
`class Kutya { /*...*/ };`
- **Névterek (namespaces)**
 - namespace kulcsszóval logikai csoportokba zárhatjuk a kódunkat
 - `namespace Allatok {`
`class Eledel {};` `//Allatok::Eledel`
`namespace Emlos { //Allatok::Emlos`
`class Kutya {};` `//Allatok::Emlos::Kutya`
`}`
`}`
 - a névterek egymásba is ágyazhatók
 - névtérnek nem kötelező nevet adni (névtelen névtér)
 - ezekkel elkerülhető, hogy két különböző könyvtárban azonos nevű osztály ütközzön

- névtér használatba vétele a using namespace <névtér neve>-el oldható meg
 - alapértelmezetten a global (::) namespace-be keres. de ezzel kibővíthető erre a névtérre is
 - névütközésre ilyenkor figyelni kell
 - ez nem érvényteleníthető, ezért header fájlalba nem szokás használni

```
class Cica{}; //Cica;
namespace Allatok{
    class Kutya{}; //Allatok::Kutya;
    class Cica{}; //Allatok::Cica;
}
using namespace Allatok; //Allatok nevter hasznalata
int main(){
    Allatok::Kutya k();
    Kutya k(); //Ugyanaz mint az elozi, de egyszerubb leirni, using namespace Allatok miatt használható
    Cica c(); //Hiba, mivel a globalba es az Allatok nevterben is van azonos nevű osztály
    ::Cica c(); //Global nevterben levo osztály
    Allatok::Cica c(); //Allatok nevterben levo osztály
};
```

Absztrakt-, Interfész- és Generikus osztályok

- az absztrakció célja, hogy elfedje a részleteket (megvalósítást) és csak a lényeges tulajdonságokat és viselkedést mutatja
 - segít arra fókuszálni, hogy mit csinál az osztály és nem arra, hogy hogyan csinálja azt
 - gyakran nem konkrét megvalósításokra, hanem viselkedési mintákra van szükségünk

Absztrakt osztályok

- **olyan osztályok amik nem példányosíthatók**, és tartalmazhatnak absztrakt metódusokat (kód nélküli deklarációkat)
 - amennyiben egy osztálynak van 1 absztrakt metódusa akkor az osztály automatikusan abstract lesz
 - előfordulhat abstract osztály absztrakt metódus nélkül is
 - tartalmazhat konstruktort és adattagokat is
- céljuk, hogy ősosztályként (szülőosztályként) szolgálni, azaz egy közös alapot adni a leszármaztatott osztályoknak, kikényszerítve bizonyos metódusok megvalósítását is
- **JAVA megvalósítás**
 - többszörös öröklődés nem megoldható absztrakt osztályokkal
 - abstract kulcsszóval jelölt osztályok
 - lehetnek megvalósított (törzzsel) ellátott metódusai és abstract kulcsszóval jelölt, törzs nélküli metódusai
 - ha már van legalább 1 absztrakt metódusa egy osztálynak akkor az egész osztálynak absztraktnak kell lennie

- **C++ megvalósítás**
 - nincs abstract kulcsszó az osztályra
 - egy osztály akkor lesz absztrakt, ha tartalmaz legalább egy **tiszta virtuális eljárást**
 - `virtual void f() = 0;`

Interfész osztályok

- **C++-ban nincs interface**, de absztrakt osztállyal amibe minden metódus tisztán virtuális (`virtual void f() = 0;`) és nincs adattagjuk megoldható ugyanolyan viselkedés mint egy JAVA interfésszel
- az interfész segítségével egy osztály váza írható le, azaz, hogy milyen metódusokkal rendelkezik, anélkül, hogy ezek implementásra kerülnének
 - JAVA 8 óta vannak default metódusok, azaz előfordulhat a metódusnak törzse, megvalósításai
- interfészek metódusokat definiálnak, amiket más osztályok implementálhatnak
 - ha az adott osztályokba legalább 1 metódusa az interfésznek nem kerül implementálásra, akkor az osztály abstract lesz
- az interfész is egy absztrakt adattípus
 - attribútumai alapból: `public`, `final` és `static`
 - nem tartalmazhat interfész példányváltozókat, mert alapból statikusok az attribútumok
 - a metódusok alapból: `public` és `abstract`
 - `final` nem lehet, mert a `final` metódusok nem írhatók felül
 - JAVA 9 óta lehet `private` metódus
 - az interfész nem tartalmaz konstruktort
- **JAVA**
 - kulcsszava az **implements**
 - többszörös öröklődés JAVA esetén interfészekkel megoldható
 - egy osztály több interfész implementálhat
 - JAVA 8 és a fölött, mert **default metódusok** szükségesek
 - ha egy osztály 2 interfészt valósít meg és ezeknek van azonos nevű default metódusa akkor az `@Override` szükséges
- **absztrakt osztály vs interfész**
 - az absztrakt osztály tartalmazhat állapotot (adattagok) és kész, implementált metódusokat is
 - JAVA 8 óta igazából interfészbe is lehetnek metódus implementációk a default metódusoknak köszönhetően
 - ha "is-a" (egyfajta) kapcsolat van, akkor absztrakt osztály
 - pl. a Kutya egy Állat
 - ha "can-do" (képes rá) kapcsolat, akkor interfész
 - pl. a Kutya tud Futni, de a Gépkecske is tud Futni/Haladni

Generikus osztályok (sablonok)

- a generikus osztályok lehetővé teszik, hogy az osztályokat típusfüggetlenül írjunk meg, azaz nem rögzítjük le előre, hogy milyen típusú adatokkal fog dolgozni
 - típus csak akkor dől el, mikor az osztályból konkrét objektumot példányosítunk
- motiváció, hogy mikor a kód ugyanaz körülbelül, de több típus merülhet fel, akkor generikus osztály nélkül minden típushoz külön osztályt kéne létrehozni, ez kódduplikáció, viszont generikus osztályt használva 1-el is megoldható
 - pl: van egy tároló osztály, mely egy értéket elment, generikusok nélkül minden típusra kellene egy külön
 - kell egy EgeszTarolo, az int-hez, egy SztringTarolo, a String-hez és egy KutyaTarolo a Kutya objektumokhoz
- előnyük a **típusbiztonság** és a **kódduplikáció elkerülése**
 - típusbiztonság = nem kell kényszerített típuskonverzió/casting
- **JAVA generikusok (generics)**
 - típusparamétereket használunk (ArrayList<T>)
 - fontos, hogy primitív típus (pl: int) nem lehet paraméter (T), csak csomagolóosztály (pl: Integer)
 - **típuseltörlés**
 - a típusinformáció csak a fordítási időben létezik, a fordítás után a típust úgymond elfelejtődik és mindent Object-ként kezelődik belül
 - szigorúbbak mint a C++-os sablonok, de futásidőben kevesebb információt hordoznak a típusról
 - ezzel kompatibilis marad a régebbi verziókkal
 - List<String> nevek = new ArrayList<>();

```
public class Doboz<T> {
    private T tartalom; // T típusú változó

    public void betesz(T dolog) {
        this.tartalom = dolog;
    }

    public T kivesz() {
        return tartalom;
    }
}

// Használat:
Doboz<Integer> szamDoboz = new Doboz<>(); //
szamDoboz.betesz(10);

Doboz<String> szovegDoboz = new Doboz<>(); /.
szovegDoboz.betesz("Hello SZTE");
```

- **C++ templates (sablonok)**

- template <typename T> kulcsszóval definiáljuk
- **kódgenerálás**
 - minden egyes típushoz (pl: int, double) amivel meghívjuk a sablont generál egy külön gépi kódot a fordítás közben
 - kódduplikálás történik
 - gyorsabb futást, de nagyobb bináris méretet eredményezhet
- C++ sablonok rugalmasabbak (lehetnek benne konstans paraméterek is, pl. tömbméret) mint a JAVA generikusok
- std::vector<int> számok;

```
template <typename T>
class Doboz {
private:
    T tartalom;
public:
    void betesz(T dolog) {
        tartalom = dolog;
    }
    T kivesz() {
        return tartalom;
    }
};

// Használat:
Doboz<int> szamDoboz;
szamDoboz.betesz(5);

Doboz<std::string> szovegDoboz;
szovegDoboz.betesz("C++ sablon");
```

VIRTUÁLIS ELJÁRÁSOK ÉS MEGVALÓSÍTÁSUK

Virtuális eljárások

- a virtuális eljárások lehetővé teszik a **dinamikus kötést** (late binding), ami a polimorfizmus alapja
 - a dinamikus kötés azt jelenti, hogy futási időben (futás közben) dől el az objektum tényleges típusa alapján, hogy melyik metódus fusson le
 - polimorfizmus pedig azt jelenti, hogy egy őosztály típusú mutatóval vagy referenciával hivatkozunk egy gyermekosztály típusú objektumra
 - a dinamikus kötésnek hála pedig a felüldefiniált metódus hívódik meg
- **kiterjeszthetőség**
 - virtuális metódusokkal az új gyermekosztályoknak adhatunk egyedi megvalósításokat és új metódusokat is, azaz kiterjeszthetjük

- **JAVA**

- nincs külön kulcsszó a virtuális metódusokra
- minden nem-static és nem-final metódus alapértelmezetten virtuális ezért a dinamikus kötés automatikus

- **C++**

- C++-ban alapértelmezésben a kötés statikus
 - statikus kötés az, ha egy őosztály típusú mutatóval hivatkozunk egy gyerekobjektumra, és meghívunk egy metódust, az őosztály metódusa fut le
- a dinamikus viselkedéshez a **virtual** kulcsszót kell használni
 - dinamikus kötés az, ha egy metódusról tudjuk, hogy azt a leszármazott osztályokban felül szeretnénk írni, az őosztályban a függvény elé kell kiírni a virtual-t
- ha egy függvény az ősből virtuális akkor ha nem írjuk ki a gyermekosztályban akkor az még virtual marad
- **virtuális destruktor**
 - ha egy osztálynak van legalább egy virtuális metódusa akkor kötelezően a destruktor virtuálissá kell tenni
 - ha egy ős típusú pointeren keresztül törölünk egy leszármazottat, virtuális destruktor nélkül csak az ős destruktor futna le, ami memóriaszivárgáshoz vezetne
- **megvalósítása**
 - a háttérben a VMT (Virtual Method Table) segítségével valósul meg:
 1. minden olyan osztályhoz, amelynek van virtuális függvénye, a fordító létrehoz egy statikus táblázatot (VMT)
 - ez tartalmazza az osztály virtuális metódusainak memóriacímeit (függvénymutatókat)
 2. ha egy gyerekosztály felülír (override) egy metódust, az ő VMT-jében az új kód címe fog szerepelni
 3. minden objektumpéldány kap egy rejtett adattagot (**vptr**, azaz Virtual Pointer) a konstruktor lefutásakor, mely az adott objektum osztályának VMT-jére mutat
 - **hívás folyamata (feloldás)**, az az mikor egy virtuális metódust hívunk (allat->hangotAd()):
 1. megkeresi az objektum vptr-ét
 2. vptr alapján eljut az osztály VMT-jéhez
 3. a táblázatból kikeresi a megfelelő metódushoz tartozó memóriacímet
 4. majd erre a címre ugrik a vezérlés (végrehatja a kódot)
 - ez a mechanizmus kis teljesítményvesztéssel jár (indirekció), de ez az ára a rugalmasságnak
 - lépcsős memóriahozzáférés (mutató követése -> táblában keresés -> ugrás)